MVP stage

Initial goals

- Finish survey feature
- Finish login and signup feature

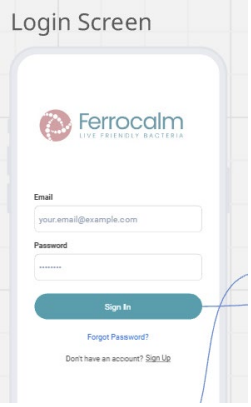
Initial plan

We planned to start in pairs with one pair doing frontend and the other doing backend. We had planned to do 2 weeks of this and then switch pairs so that each person could learn Flutter and Springboot. This didn't end up working as one of our team members fell ill and had an injury which meant a lack of meetings so this extended the time in which we were doing frontend and backend teams. Then with reading week another one of our teammates went away so we weren't able to meet which made us commit to only having a frontend and backend team.

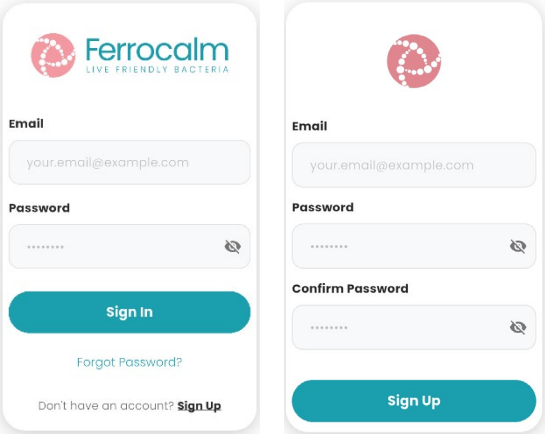
Login and signup feature

We setup Flutter and Springboot into our repo with a template from which we built our application.

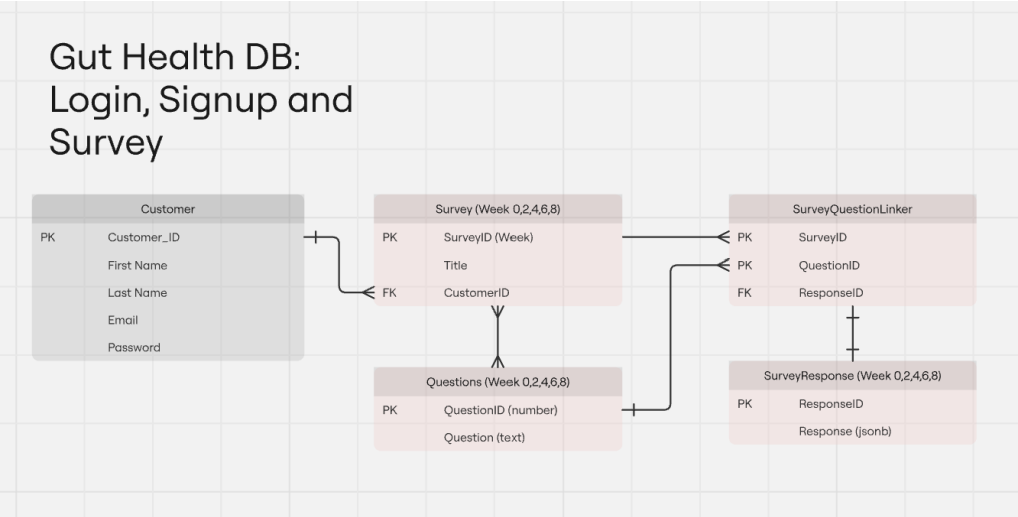
The frontend team made the UI design for the login page on miro:



We learned Flutter and created the pages on Flutter, dart was very intuitive to learn and was easily picked up so the pages were made in good amount of time:



The backend team worked on designing the database:



We also setup the Springboot app in our repo with files for login and signup.

We had a meeting with our client and they mentioned that they would like it if we could use their existing customer data from their shopify website. This was a setback as it wasn't mentioned to us before and we had to rethink our database. We talked to Sarah about this and she said we wouldn't be able to do this until a further stage and we couldn't use real customer data. This caused us a week of a lack of progress where we only did validation for both login and signup on the frontend side.

After this we solely focussed on the survey page as there wasn't enough time to finish the login and signup page along with the survey page.

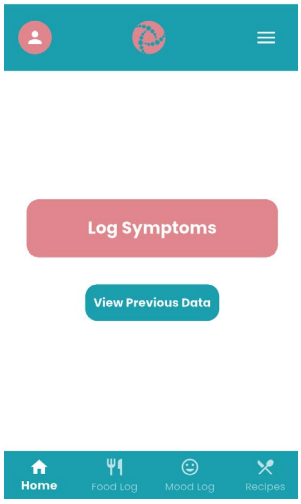
Survey feature

For this feature our goals had now changed to just being able to connect frontend and backend for one survey question.

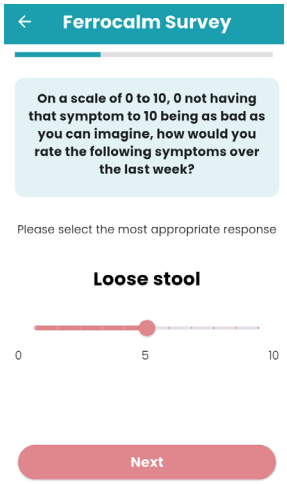
Niyor worked on the home page and Yuan worked on the survey page on the frontend side building the pages on Flutter.

Ananya and Lingyi worked on the Springboot and Database(Postgres) side making all the java files and connecting the database into the app.

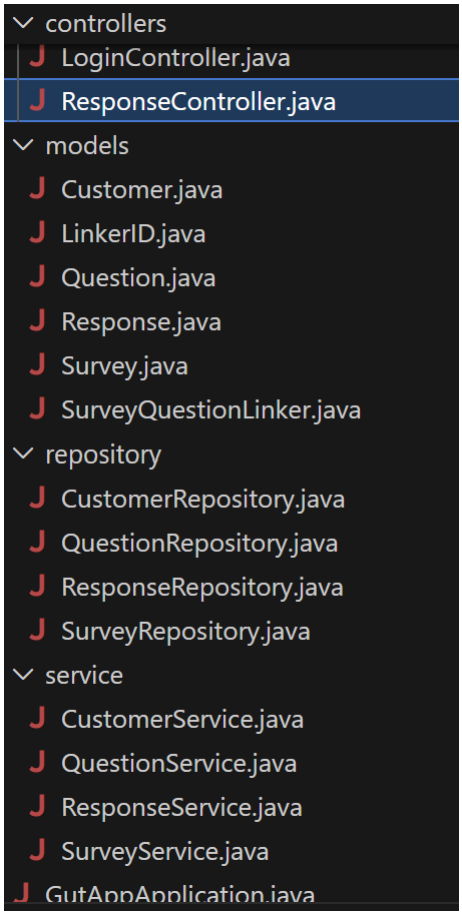
The home page connected to the survey page and all the other buttons stayed unimplemented:



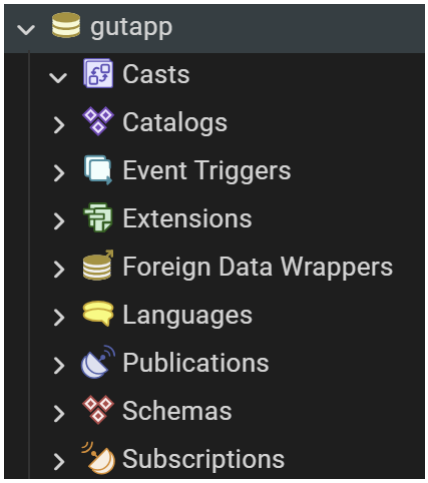
The survey page included one question that collected a response as a JSON datatype:



The java files were created:



And we decided to implement a local Postgres database as there wasn't enough time to host it on AWS.



This was connected via the application.properties file:

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.properties.hibernate.format_sql=true
```

And a local .env file containing the url, username and password which is in the gitignore so not available for everyone to see.

In theory the survey response would be collected here:

```
    @PostMapping("/submit")
    public Response saveResponse(@RequestBody Response response) {
        return responseService.createResponse(response);
    }
```

However, there were some issues:

- Data type: the save response expects a response class which contains {attributes: {__,__}} so we had to wrap our response with attributes in the dart file
- Port issue: when running on an emulator the local host port is different: instead of localhost:5333 it would be //10.0.2.2:5333
- Server port issue: instead of it being the Postgres database port, it was the springboot server port so it was 8080 instead of 5433

There were also issues with running the backend:

The springboot app wasn't staying alive due to some errors. One of the errors was because of some of the java files not having the correct attributes and types. Another was because of a java file not having a constructor. The final one was because of some machines not being able to read the .env file.

These were easily fixable.

We then created CI for both frontend and backend and also wrote tests for both.

The backend CI didn't work because it wasn't stated to run on the dev branch.

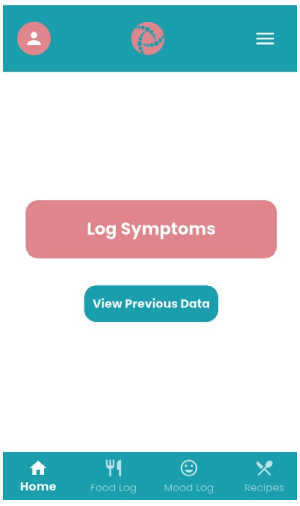
When this was fixed an issue arose which was that the backend tests needed a database to run on so the H2 package was imported for that.

Another issue was that one of the test files were empty so removing that file fixed all the issues and all tests passed.

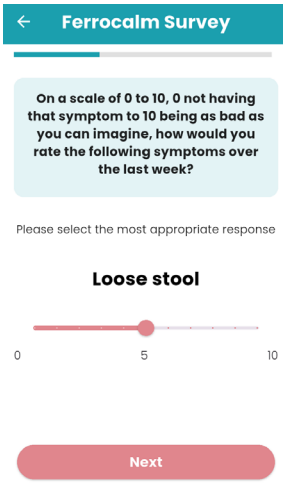
MVP stage demonstrated

This is what our app does so far:

- Run ./mvnw spring-boot:run on a terminal in the guthealth directory
- Run the flutter app on an emulator
- Takes the user to the home page



- When log symptoms is pressed it goes to the survey page



- When next is pressed a response is sent from Flutter:

```
I/flutter (15677): All answers (JSON): {"symp_loose_stool":5.0}
I/flutter (15677): Saved successfully: {"responseID":3,"attributes":{"symp_loose_stool":5.0},"response":null}
```

- This sends a response to the springboot server which is connected to the postgres database

```
2025-11-20T16:47:05.245Z  INFO 56176 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Sprin
g DispatcherServlet 'dispatcherServlet'
2025-11-20T16:47:05.247Z  INFO 56176 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servl
et 'dispatcherServlet'
2025-11-20T16:47:05.250Z  INFO 56176 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initiali
zation in 2 ms
Hibernate:
    insert
    into
        response
        (attributes)
    values
        (?)
```